Pontificia Universidad Javeriana
Departamento de Ingeniería de Sistemas

Administración de Bases de Datos (DBA) – 2026-30

Proyecto final

Entrega 1: Diseño del esquema NoSQL y plan de implementación

Potencial solar en techos de los municipios PDET

Maria Camila Beltran

Jorge Fortich

Jose Eduardo Valeriano

Samuel Osorio Rodríguez

Profesor: Andrés Oswaldo Calderón Romero, Ph.D.

2 de noviembre de 2026

Índice

1. Contexto y objetivo **2**

2. Decisiones tecnológicas **2**

3. Plan de implementación **3**

4. Diseño del esquema **4**

5. Diccionario de datos **5**

5.1. `municipios` 5

5.2. `edificios_google` y `edificios_microsoft` 6

5.3. `fuentes` 6

5.4. `resultados` 7

5.5. `bitacora_cargas` 7

5.6. Índices 7

1. Contexto y objetivo

El proyecto estima, para cada uno de los **170 municipios PDET**, cuántas edificaciones hay y cuál es el área total de techos disponible para paneles solares. Para eso compara dos fuentes abiertas de huellas de edificios y usa una base de datos NoSQL con índices espaciales.

La Oficina de Investigación e Información de la UPME busca municipios candidatos para proyectos piloto de granjas solares. A mayor área de techos en un municipio, mayor potencial de captación. La lista oficial de municipios viene en `MunicipiosPDET.xlsx`: 170 municipios en 16 subregiones PDET y 19 departamentos [1].

Esta primera entrega fija las bases del proyecto:

- **Plan de implementación**: la metodología completa, desde la descarga de los datos hasta el informe final.
- **Diseño del esquema y modelo de datos**: el diagrama E-R de las colecciones y el diccionario de datos.

Las entregas siguientes construyen sobre este diseño: límites PDET (semana 2), carga de edificios (semana 3) y análisis e informe final (semana 4).

2. Decisiones tecnológicas

Usaremos **MongoDB** con índices `2dsphere` y compararemos **Google Open Buildings v3** con **Microsoft Building Footprints**. Todo el flujo corre en Python dentro de Docker para que sea reproducible.

Tabla 1: Decisiones tecnológicas del proyecto.

Decisión	Elección	Justificación
Motor NoSQL	MongoDB 8 (modelo documental)	Guarda geometrías en GeoJSON de forma nativa, tiene índices `2dsphere` y operadores `$geoWithin` y `$geoIntersects` [5]. Su pipeline de agregación con `$group` y `$merge` calcula los totales por municipio dentro de la base. Puede escalar horizontalmente con sharding.
Fuente de edificios 1	Google Open Buildings v3 [3]	Cubre América Latina con imágenes de 50 cm (inferencia de mayo de 2023). Trae `area_in_meters` y `confidence` (de 0,65 a 1,0), lo que permite validar nuestras áreas y hacer análisis de sensibilidad.
Fuente de edificios 2	Microsoft Building Footprints [4]	Imágenes de 2014 a 2021 con licencia ODbL. Está en GeoParquet particionado por país (`RegionName`) y por quadkey de nivel 9, así que se puede descargar solo Colombia. Solo trae la geometría, por lo que el área se calcula.
Límites municipales	MGN 2025 del DANE, nivel municipio [2]	Es la fuente oficial que exige el enunciado. Se filtra con los 170 códigos de `MunicipiosPDET.xlsx`.
Sistemas de referencia	Almacenamiento en WGS84 (EPSG:4326); áreas en MAGNA-SIRGAS Origen Nacional (EPSG:9377)	GeoJSON y `2dsphere` exigen longitud y latitud en WGS84. Las áreas se calculan en una proyección plana oficial de Colombia para que queden en metros cuadrados.
Herramientas	Python 3.12, GeoPandas, Shapely, PyMongo, DuckDB, Docker Compose, GitHub	GeoPandas valida y reproyecta. DuckDB lee los Parquet y filtra antes de cargar. PyMongo carga por lotes. Docker Compose levanta la misma base en cualquier máquina.

3. Plan de implementación

El flujo tiene cinco fases que corren con un solo comando desde el repositorio (`make all`). Cada fase lee sus parámetros de `config.yaml` y deja su registro en la colección `bitacora_cargas`.

1. **Infraestructura (semana 1)**
   - Levantar MongoDB 8 con Docker Compose. La base de datos, los volúmenes y las credenciales quedan versionados en el repositorio, sin secretos.
   - Definir la estructura del repositorio: `src/` (scripts por fase), `config.yaml`, `notebooks/` (EDA), `docs/` (informes) y `Makefile`.
   - Crear las colecciones con validación de esquema (`$jsonSchema`) a partir del diccionario de la sección 5.
2. **Límites PDET (semana 2)**
   - Descargar el MGN 2025 del geoportal del DANE con un script que guarda la URL, la fecha y el checksum del archivo.
   - Leer la capa municipal y cruzarla con `MunicipiosPDET.xlsx`. Los códigos DANE del Excel vienen como números, así que los de Antioquia pierden el cero inicial (5045 en vez de 05045): se rellenan a cinco dígitos antes del cruce.
   - Verificar que los 170 municipios crucen. Reparar las geometrías inválidas con `make_valid`, reproyectar a WGS84 y revisar que cada documento quede por debajo del límite de 16 MB de MongoDB.
   - Cargar los municipios en `municipios` y crear el índice `2dsphere` sobre `geometry`. Como evidencia del índice se usa `explain()`, que muestra la etapa `IXSCAN`.
3. **Huellas de edificios (semana 3)**
   - Descargar solo lo necesario: las teselas de Google que tocan Colombia y las particiones `RegionName=Colombia` de Microsoft.
   - Prefiltrar con DuckDB o GeoPandas por el rectángulo envolvente de los municipios PDET, para no cargar todo el país.
   - Por cada edificio: validar la geometría, calcular el centroide, calcular `area_m2` en la proyección EPSG:9377 y generar `geom_hash` para detectar duplicados entre particiones.
   - Hacer la carga masiva con `insert_many(ordered=False)` en lotes de tamaño configurable. Los índices se crean después de cargar, porque construirlos sobre una colección llena es más rápido que mantenerlos inserción por inserción.
   - Hacer un EDA por fuente: número de edificios, distribución de áreas, distribución de `confidence` (Google) y comparación entre `area_in_meters` y nuestra `area_m2`.
4. **Análisis espacial (semana 4)**
   - Asignar cada edificio a un solo municipio con la **regla del centroide**: el edificio pertenece al municipio que contiene su centroide (`$geoWithin` sobre `centroide`). Así un techo que cruza el límite entre dos municipios no se cuenta dos veces.
   - Agregar por municipio y fuente con `$group` y escribir en `resultados` con `$merge`. Las métricas son: número de edificios, área total de techos, área media, densidad (edificios/km^2) y porcentaje del municipio cubierto por techos.
   - Parametrizar el umbral de confianza de Google (por defecto 0,65, el mínimo del dataset) y repetir el cálculo con 0,70 y 0,75 para medir la sensibilidad.

- Validar con un cruce espacial independiente en GeoPandas sobre una muestra de municipios. Los conteos deben coincidir con los de MongoDB.

5. **Resultados e informe (semana 4)**

- Exportar tablas por municipio y por subregión. Hacer mapas coropléticos del área de techos por municipio para cada fuente y un mapa de la diferencia entre fuentes.
- Proponer un ranking de municipios candidatos para los pilotos solares, con la discusión de por qué difieren las fuentes.

El área de techo de cada edificio se aproxima con el área de su huella en planta, que es lo que entregan ambos datasets. Para cada municipio m y fuente f:

$$A_{m,f} = \sum_{e \in E_f,\ \text{centroide}(e) \in m} \text{area}_{9377}(e) \tag{1}$$

donde E_f es el conjunto de edificios de la fuente f y $\text{area}_{9377}(e)$ es el área de la huella e calculada en EPSG:9377.

4. Diseño del esquema

El esquema tiene cinco colecciones. Las centrales son los municipios y los edificios, que se relacionan por una consulta espacial: cada edificio pertenece al municipio que contiene su centroide (figura 1).

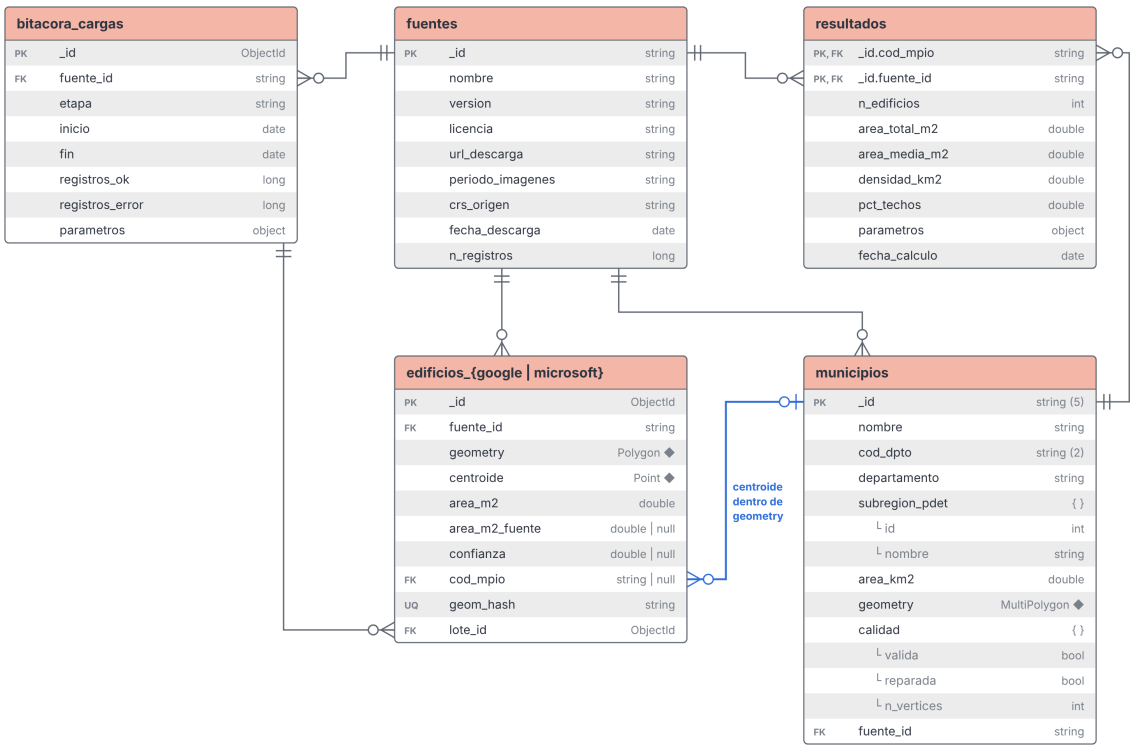


Figura 1: Diagrama E-R del esquema en MongoDB.

El E-R se adapta al modelo documental de MongoDB y usa notación pata de gallo. Cada relación es una referencia por `_id` y conecta el campo que referencia (FK) con la llave del documento referenciado (PK). Los símbolos en los extremos de cada línea indican la cardinalidad:

- **Dos barras** (‖): exactamente uno.
- **Círculo y barra**: cero o uno.
- **Círculo y pata de gallo**: cero o muchos.
- **Línea azul**: relación espacial. No es una llave foránea común: `cod_mpio` se llena con `$geoWithin`, asignando cada edificio al municipio que contiene su centroide.

El símbolo ⧫ indica un índice `2dsphere`, y los campos indentados bajo `subregion_pdet` y `calidad` son subdocumentos embebidos.

- **Subregión PDET embebida en el municipio.** Es un dato pequeño, no cambia y siempre se lee junto con el municipio.
- **Edificios referenciados, no embebidos.** Un municipio puede tener miles de edificios, y embebidos superarían el límite de 16 MB por documento. Cada edificio guarda `cod_mpio` como referencia.
- **Una colección por fuente de edificios.** Las dos tienen el mismo esquema, pero así se pueden cargar, reindexar o borrar por separado y comparar sin mezclar datos.
- **Centroide guardado aparte.** La asignación solo necesita un punto, e indexar un punto es más barato que indexar un polígono.
- **Resultados precalculados.** `resultados` guarda un documento por municipio y fuente, así los mapas y el informe no repiten la consulta espacial.
- **Bitácora de cargas.** Cada ejecución queda registrada con sus parámetros, como evidencia de reproducibilidad.

5. Diccionario de datos

Las cinco colecciones viven en la base `upme_solar`. Todas las geometrías están en GeoJSON con coordenadas `[longitud, latitud]` en WGS84, y todas las áreas están en m^2 calculados en EPSG:9377.

5.1. `municipios`

Campo	Tipo	Descripción	Origen
`_id`	string	Código DANE del municipio, con ceros a la izquierda (`05045`)	MGN / Excel PDET
`nombre`	string	Nombre del municipio	MGN
`cod_dpto`	string	Código DANE del departamento	MGN
`departamento`	string	Nombre del departamento	MGN
`subregion_pdet`	objeto {id: int, nombre: string}	Subregión PDET (embebida, una de 16)	Excel PDET
`area_km2`	double	Área del municipio	Calculado (EPSG:9377)
`geometry`	GeoJSON Polygon o MultiPolygon	Límite municipal	MGN (reproyectado)
`calidad`	objeto {valida: bool, reparada: bool, n_vertices: int}	Resultado de la validación de la geometría	Calculado
`fuente_id`	string	Referencia a `fuentes` (`mgn`)	Calculado

5.2. `edificios_google` y `edificios_microsoft`

Las dos colecciones tienen el mismo esquema. Los campos que una fuente no trae quedan en `null`.

Campo	Tipo	Descripción	Origen
`_id`	ObjectId	Identificador interno	MongoDB
`fuente_id`	string	Referencia a `fuentes` (`google` o `microsoft`)	Calculado
`geometry`	GeoJSON Polygon	Huella del edificio	Fuente
`centroide`	GeoJSON Point	Centroide de la huella; base de la regla de asignación	Calculado
`area_m2`	double	Área de la huella	Calculado (EPSG:9377)
`area_m2_fuente`	double o null	Área reportada por la fuente (`area_in_meters` de Google)	Fuente
`confianza`	double o null	Confianza del modelo, de 0,65 a 1,0 (solo Google)	Fuente
`cod_mpio`	string o null	Referencia a `municipios`; se llena en la fase 4	Calculado
`geom_hash`	string	Hash de la geometría para detectar duplicados	Calculado
`lote_id`	ObjectId	Referencia a la carga que trajo el registro (`bitacora_cargas`)	Calculado

5.3. `fuentes`

Campo	Tipo	Descripción
`_id`	string	`google`, `microsoft` o `mgn`
`nombre`	string	Nombre completo del dataset
`version`	string	Versión (v3, fecha de proceso de Microsoft, MGN2025)
`licencia`	string	CC BY-4.0, ODbL, etc.
`url_descarga`	string	Origen exacto de los archivos
`periodo_imagenes`	string	Fechas de las imágenes satelitales
`crs_origen`	string	Sistema de referencia en que llegan los datos
`fecha_descarga`	date	Cuándo se descargó
`n_registros`	long	Registros cargados en la base

5.4. `resultados`

Campo	Tipo	Descripción
`_id`	objeto {cod_mpio, fuente_id}	Llave compuesta: un documento por municipio y fuente
`n_edificios`	int	Edificios cuyo centroide cae en el municipio
`area_total_m2`	double	Suma de `area_m2`
`area_media_m2`	double	Área promedio por edificio
`densidad_km2`	double	Edificios por km^2 del municipio
`pct_techos`	double	Área de techos sobre el área del municipio, en %
`parametros`	objeto	Umbral de confianza, regla de asignación y versión del código
`fecha_calculo`	date	Cuándo se ejecutó

5.5. `bitacora_cargas`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador de la ejecución
`fuente_id`	string	Referencia a `fuentes`
`etapa`	string	`descarga`, `limpieza`, `carga`, `indices` o `analisis`
`inicio`, `fin`	date	Duración de la etapa
`registros_ok`, `registros_error`	long	Resultado de la etapa
`parametros`	objeto	Copia de la configuración usada

5.6. Índices

Colección	Índice	Para qué
`municipios`	`2dsphere` sobre `geometry`	Consultas espaciales sobre los límites
`municipios`	`subregion_pdet.id`	Resúmenes por subregión
`edificios_*`	`2dsphere` sobre `centroide`	Asignación con `$geoWithin` (consulta principal)
`edificios_*`	`2dsphere` sobre `geometry`	`$geoIntersects` para analizar edificios en los límites
`edificios_*`	`cod_mpio`	Agregación por municipio después de la asignación
`edificios_*`	`geom_hash` (único)	Evitar duplicados entre particiones
`resultados`	`_id` compuesto	Búsqueda por municipio y fuente

Referencias

[1] Agencia de Renovación del Territorio, *Municipios PDET* (`MunicipiosPDET.xlsx`, carpeta de recursos del curso). https://www.renovacionterritorio.gov.co/central-pdet/que-son-los-pdet

[2] DANE, *Marco Geoestadístico Nacional – Descarga de datos geoestadísticos*, 2025. https://geoportal.dane.gov.co/servicios/descarga-y-metadatos/datos-geoestadisticos/?cod=111

[3] Google Research, *Open Buildings V3 Polygons*, catálogo de Earth Engine. https://developers.google.com/earth-engine/datasets/catalog/GOOGLE_Research_open-buildings_v3_polygons

[4] Microsoft, *ms-buildings: Global ML Building Footprints*, Planetary Computer. https://planetarycomputer.microsoft.com/dataset/ms-buildings

[5] MongoDB, Inc., *Query a 2dsphere Index*, documentación de MongoDB. https://www.mongodb.com/docs/current/tutorial/query-a-2dsphere-index/